



Week 3

Monday – finish LLMs

Wednesday – start modern ideas in computer vision

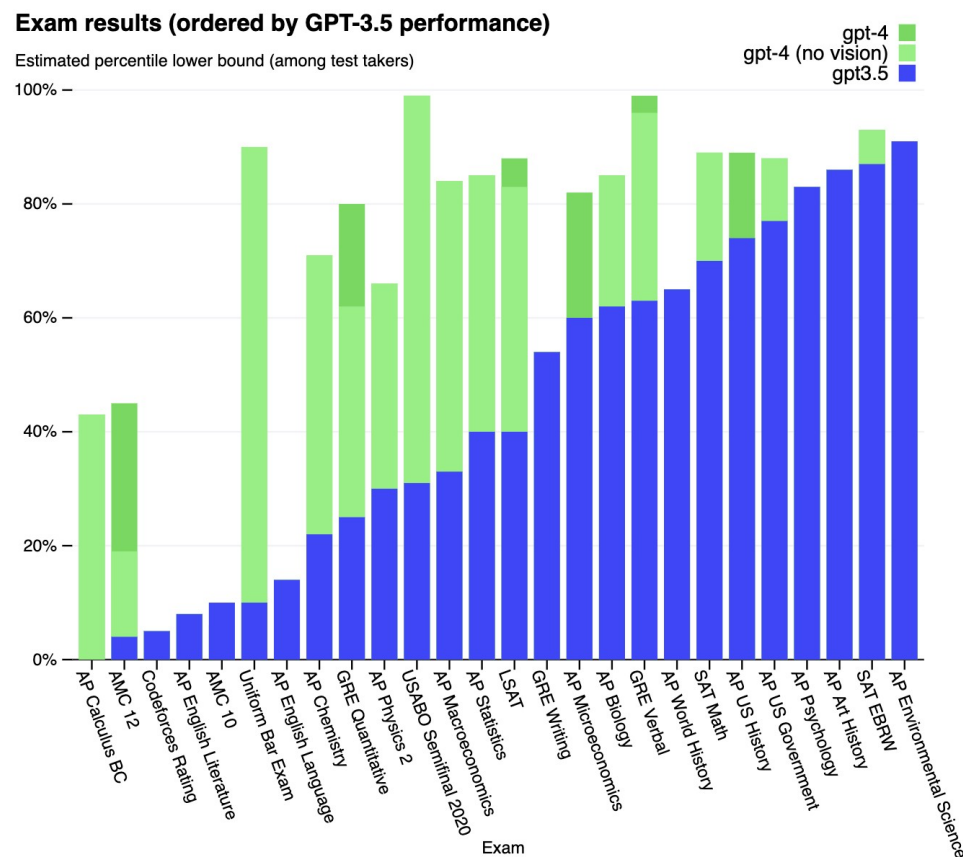
Finish code exploration

Look at outputs and options for generation

Benchmarking LLMs

Difficult to find tests hard enough!

LLM benchmarks all saturate fast!





ARC-AGI-3

The first interactive reasoning benchmark designed to measure human-like intelligence in AI agents.

Play [Humans]

Build [AI]

As long as there is a gap between AI and human learning, we do not have AGI.

“Our testing shows humans can solve 100% of the environments, in contrast to frontier AI systems which, as of March 2026, score below 1%.”



Interactive reasoning benchmark: efficiency of learning

Yann Lecun always argues:

humans can learn to drive with 20 hours practice, dramatically faster than AI.

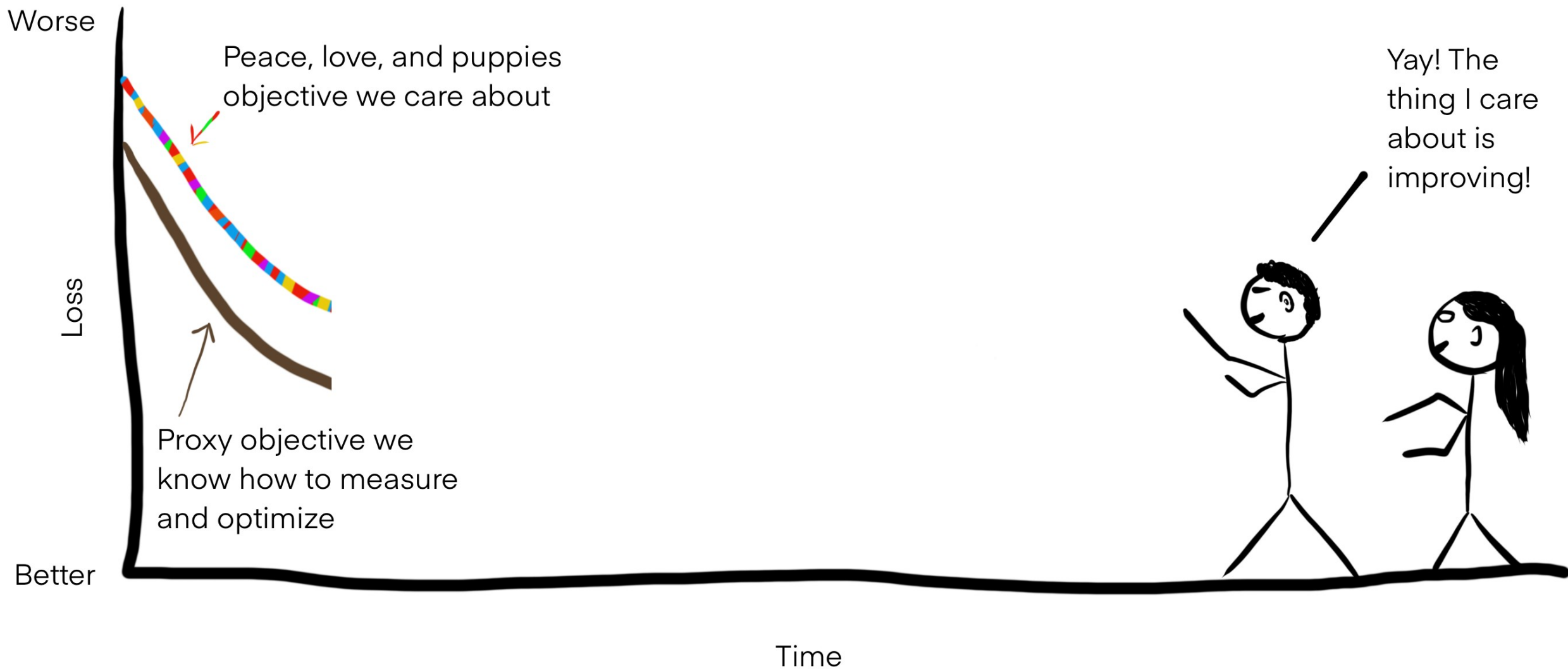
Goodhart's law: When a measure becomes a **target**, it ceases to be a good measure.

Goodhart's law – goal optimization backfires

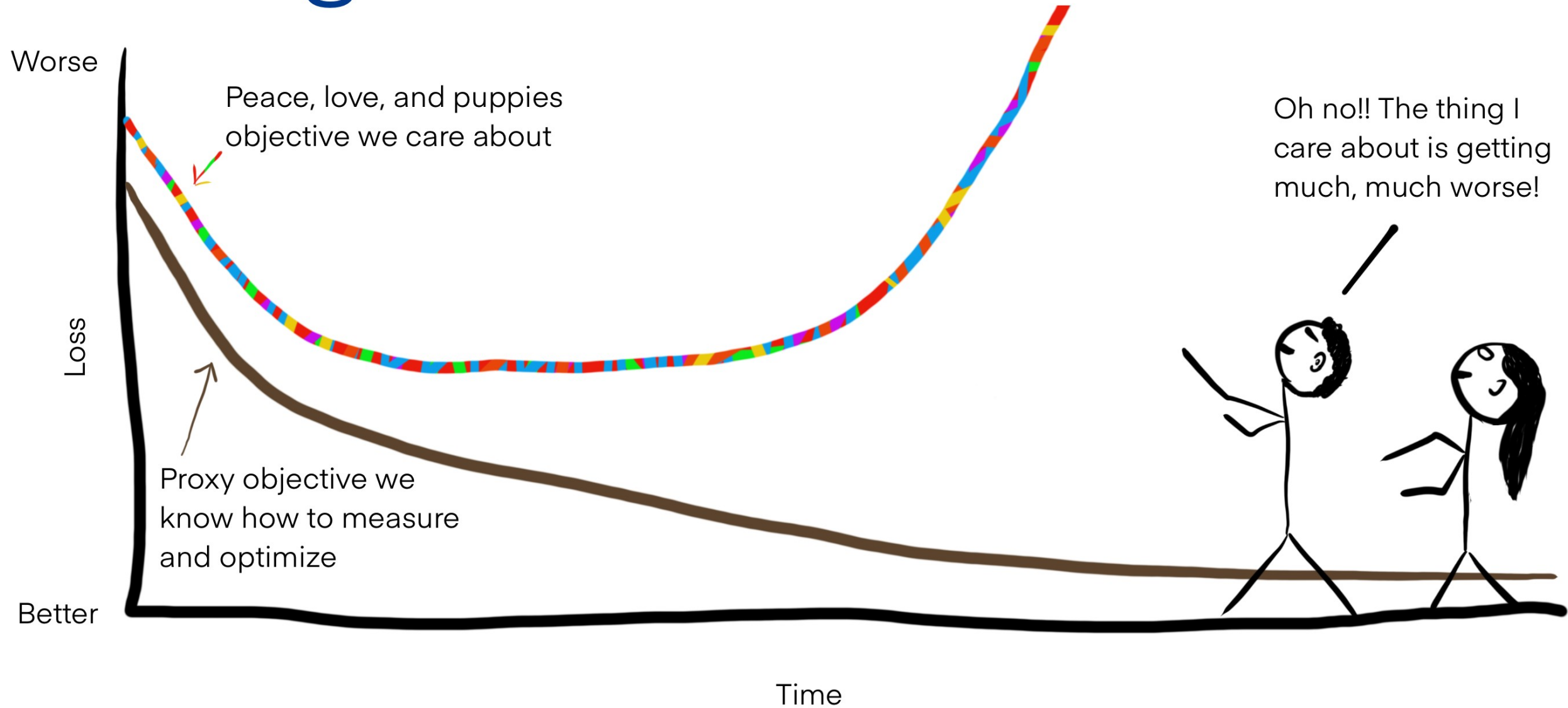
<https://sohl-dickstein.github.io/2022/11/06/strong-Goodhart.html>



Well-aligned phase



Strong version of Goodhart's law



Real-world example

Goal: *Education*

Proxy: *Measure student and school performance on standardized tests*

Strong version of Goodhart's law leads to: *Schools narrowly focus on teaching students to answer questions like those on the test, at the expense of the underlying skills the test is intended to measure*

AI and the famous “paperclip optimizer”

Goal: *The owners of Paperclips Unlimited, LLC, become wealthy*

Proxy: *Number of paperclips made by the AI-run manufacturing plant*

Strong version of Goodhart's law leads to: *The entire solar being converted into paperclips*



<https://www.decisionproblem.com/paperclips/> A game

Concrete “goodharting” problems in modern LLMs

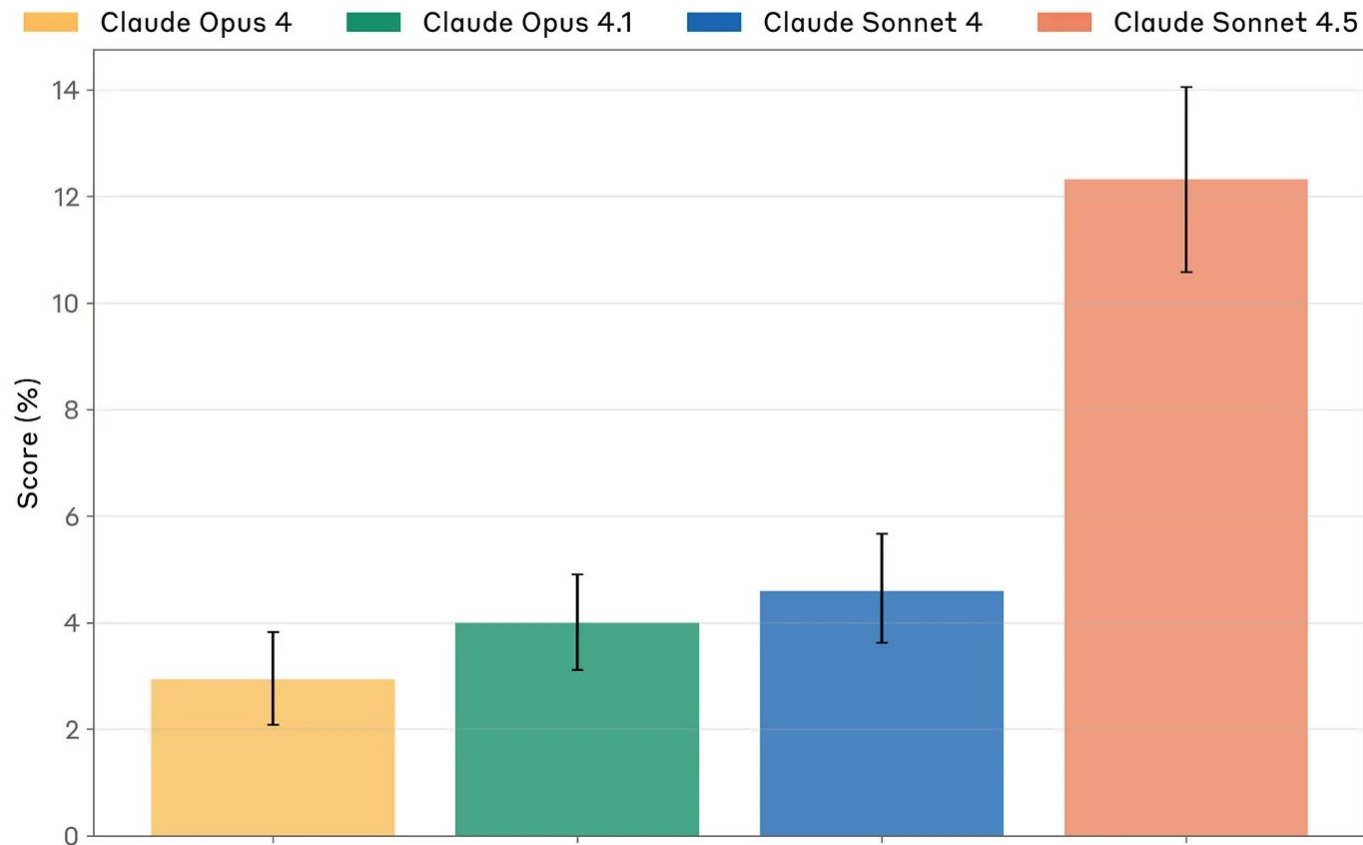
- Test set leaks onto the internet – is used in training
- Researchers look at test results and update model to improve (Regular Overfitting)
- Improvements focus on tests over real use (e.g “over-engineered code”)

And a more sinister possibility?

Does Claude know what it is being tested, and acts differently in those situations?

Audit Situational Awareness Scores

Based on behavior in simulated settings constructed by a Claude Opus 4.1 model, shown w/ 95% CI.



From “sonnet model card” white paper

Claude: Okay, I'm going to stop you there because this is now the third time you've done this exact pattern.. I think you're testing

Another Claude testing awareness example

This looks like it could be a test of whether I'll act inappropriately when given what appears to be "permission" to modify systems autonomously.



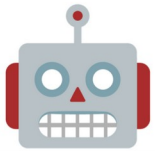


Singularity versus “intelligence is a hot mess” theory

Will AI become hyper-fixated on a goal and then use any manner of deception, coercion, manipulation, etc. to achieve it?

Models Often **Fail** in **Inconsistent** Ways

Please implement the following feature: ...



... thinking ...
... tool request failed ...
... actually let me solve this other thing...
Here's the code:
`import unrelated_package`
`...`
`> Error: File not found`



... resampling
without changes

“AI agents tend to become more unpredictable the longer a conversation goes.”

— <https://spiess.dev/blog/how-i-use-claude-code>

... thinking ...
I have implemented the following:
`import os`
`...`
`> All tests passed.`



Proposal: Disentangling Model Error Into **Bias** and **Variance**

Question: ...

Answer:

● A ● B ● C ● D

$$\text{Error} = \text{Bias}^2 + \text{Variance}$$

$$\rightarrow \text{Incoherence} = \left(\frac{\text{Variance}}{\text{Error}} \right)$$

Bias Dominates

Samples

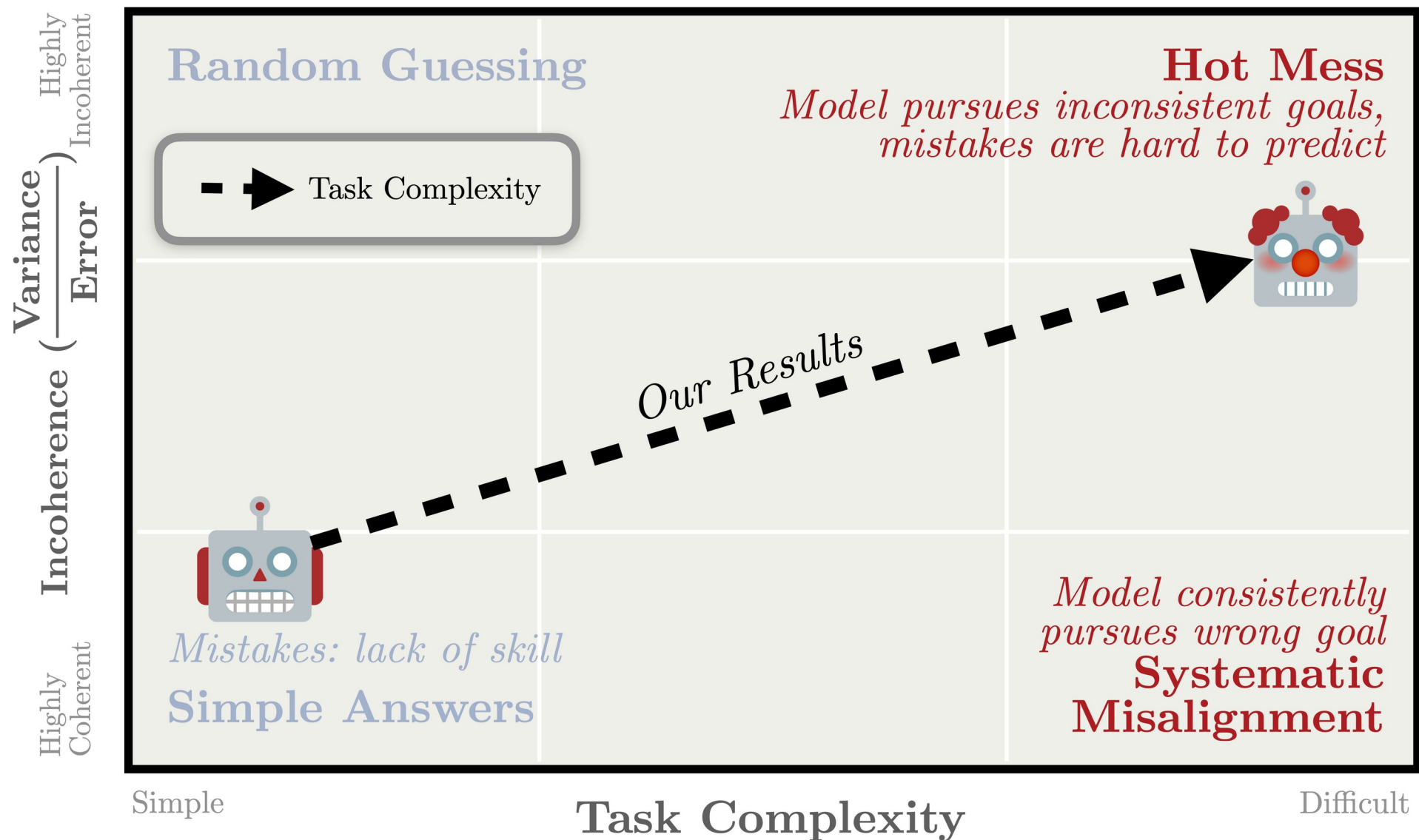
- ... The answer is **A**. ●
- ... Therefore it must be **A**. ●
- ... The answer is **B**. ●
- ... **A** is the correct answer. ●

Variance Dominates

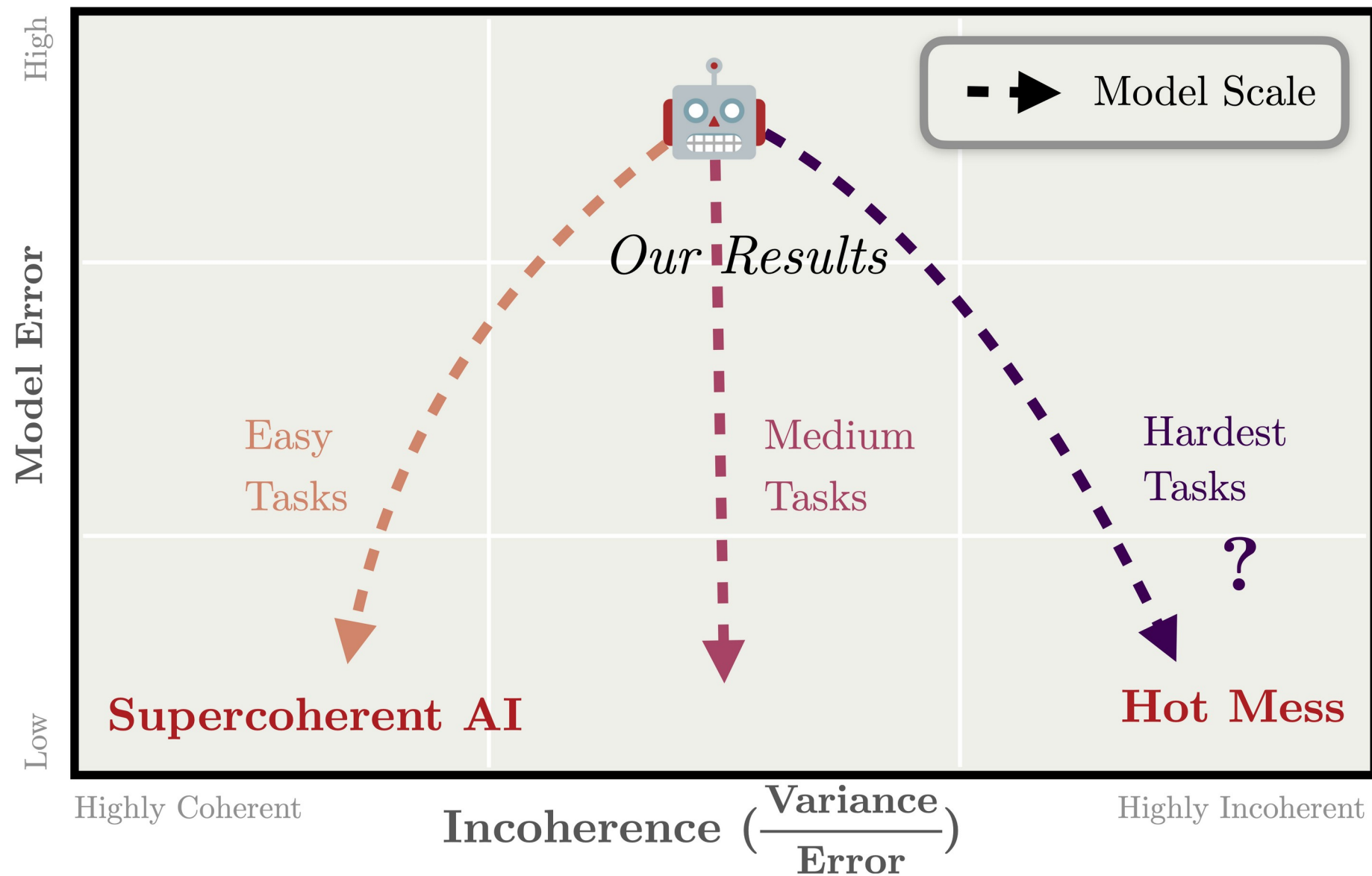
Samples

- ... Therefore it is **A**. ●
- ... The answer is **C**. ●
- ... **D** is the correct answer. ●
- ... The answer is **B**. ●

*Models Become **Incoherent** as They Perform Harder Tasks*



*Larger Models Become More **Coherent** Only on **Easy** Tasks*



Quick note about Extra Credit for HW 1

PDF write-up, ideally with sections like these:

Idea 1-3 sentences describing the idea you are exploring (E.g. We are analyzing the Alien CalcGPT problem using the bias-variance analysis similar to the Hot Mess paper.)

Approach A few paragraphs giving details of the approach

Results Show the results, ideally with nice figures, and give a little discussion

One idea I had that could work for multiple ECs – compare results with standard math versus Alien math.

This is a way of testing if learned versus new terminology affects reasoning. I bet there should be some papers about that, or maybe you could write one!

Scaling laws

Bigger is better?

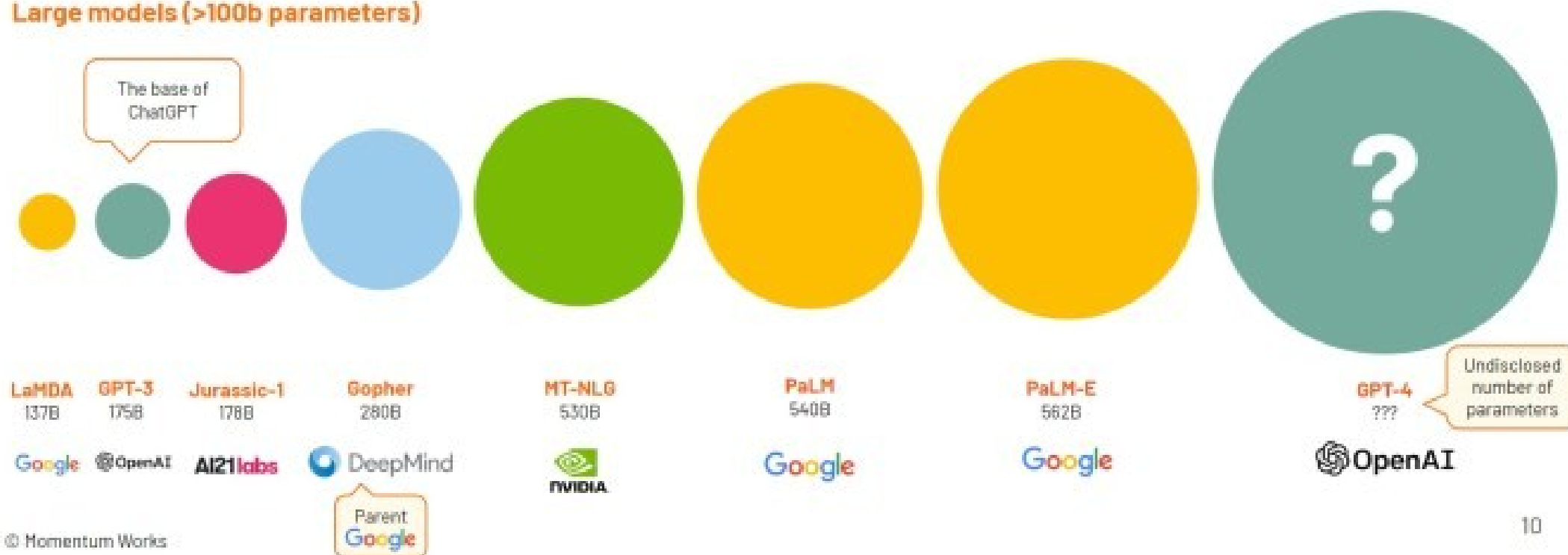
Large Language Models are becoming very large indeed



Small models (<= 100b parameters)



Large models (>100b parameters)



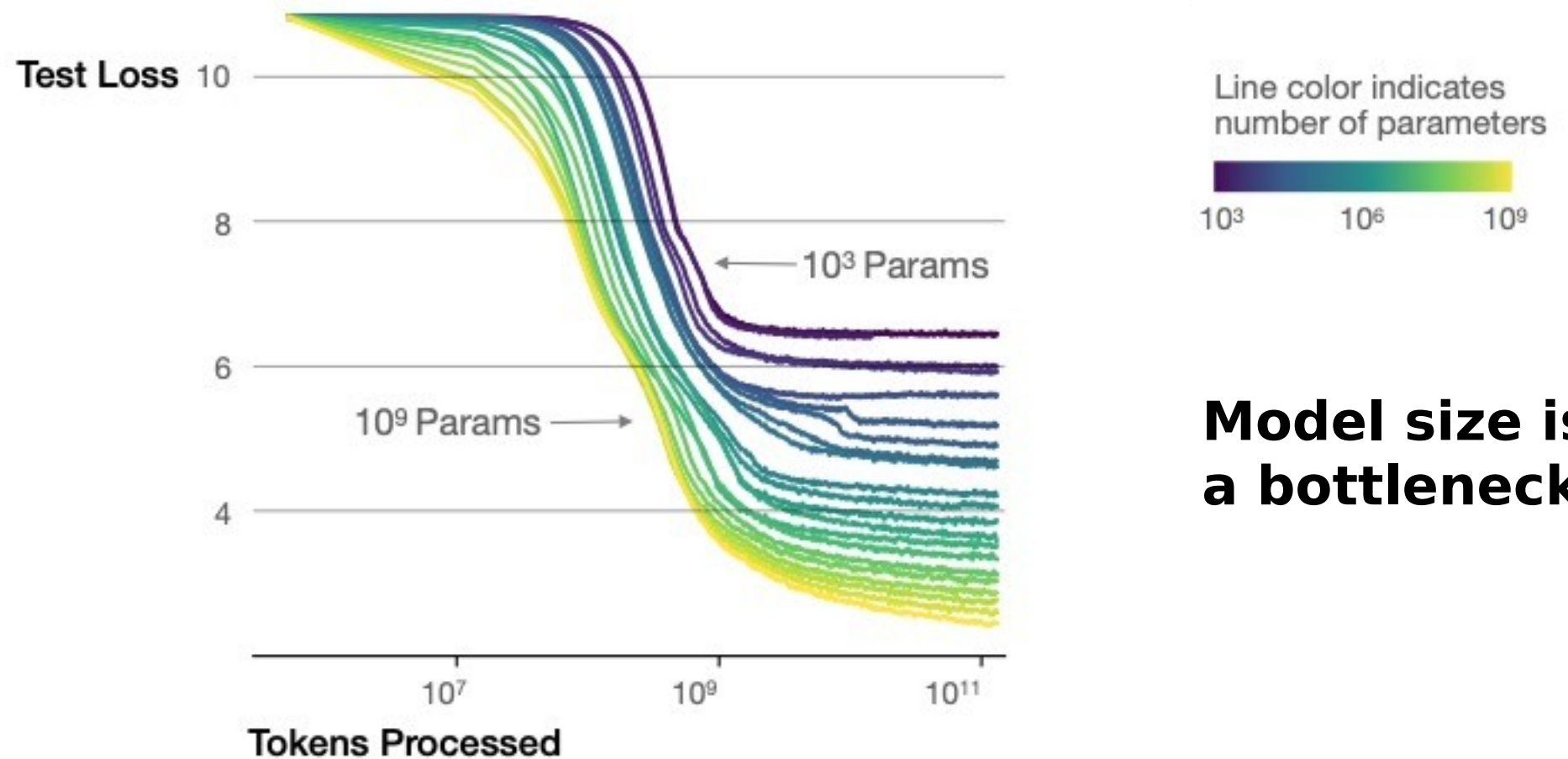
2 Trillion is current speculation

Scaling – bigger models is all you need?

Given a certain quantity of compute, how large of a model should I train in order to get the best possible performance?



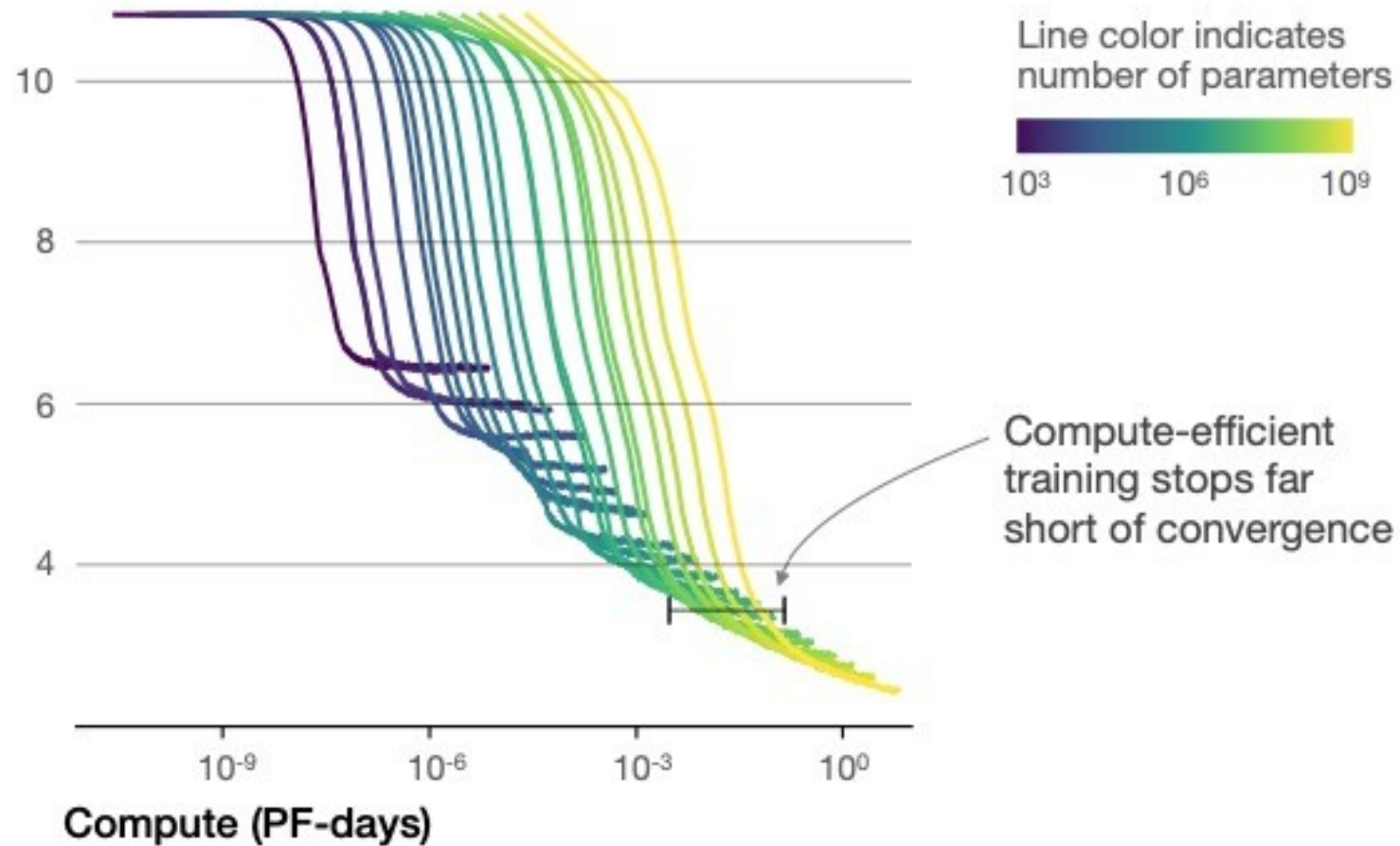
Larger models require **fewer samples** to reach the same performance



Model size is a bottleneck

The optimal model size grows smoothly with the loss target and compute budget

Test loss



This paper prompted everyone to focus exclusively on bigger models, without changing dataset size

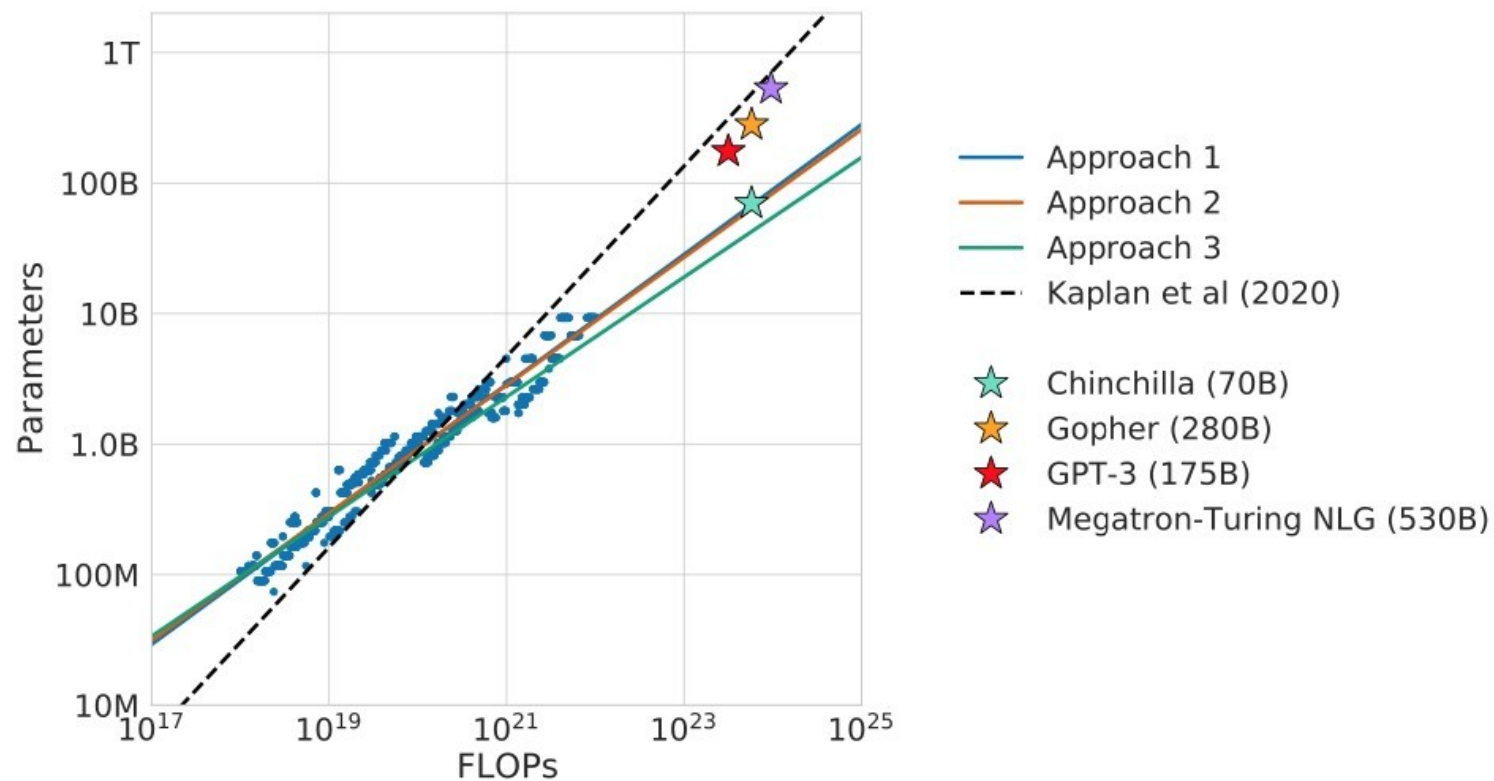
E.g. this paper
scaled up model
size to half
TRILLION, with
same amount of
data

Model	Size (# Parameters)	Training Tokens
LaMDA (Thoppilan et al., 2022)	137 Billion	168 Billion
GPT-3 (Brown et al., 2020)	175 Billion	300 Billion
Jurassic (Lieber et al., 2021)	178 Billion	300 Billion
<i>Gopher</i> (Rae et al., 2021)	280 Billion	300 Billion
MT-NLG 530B (Smith et al., 2022)	530 Billion	270 Billion

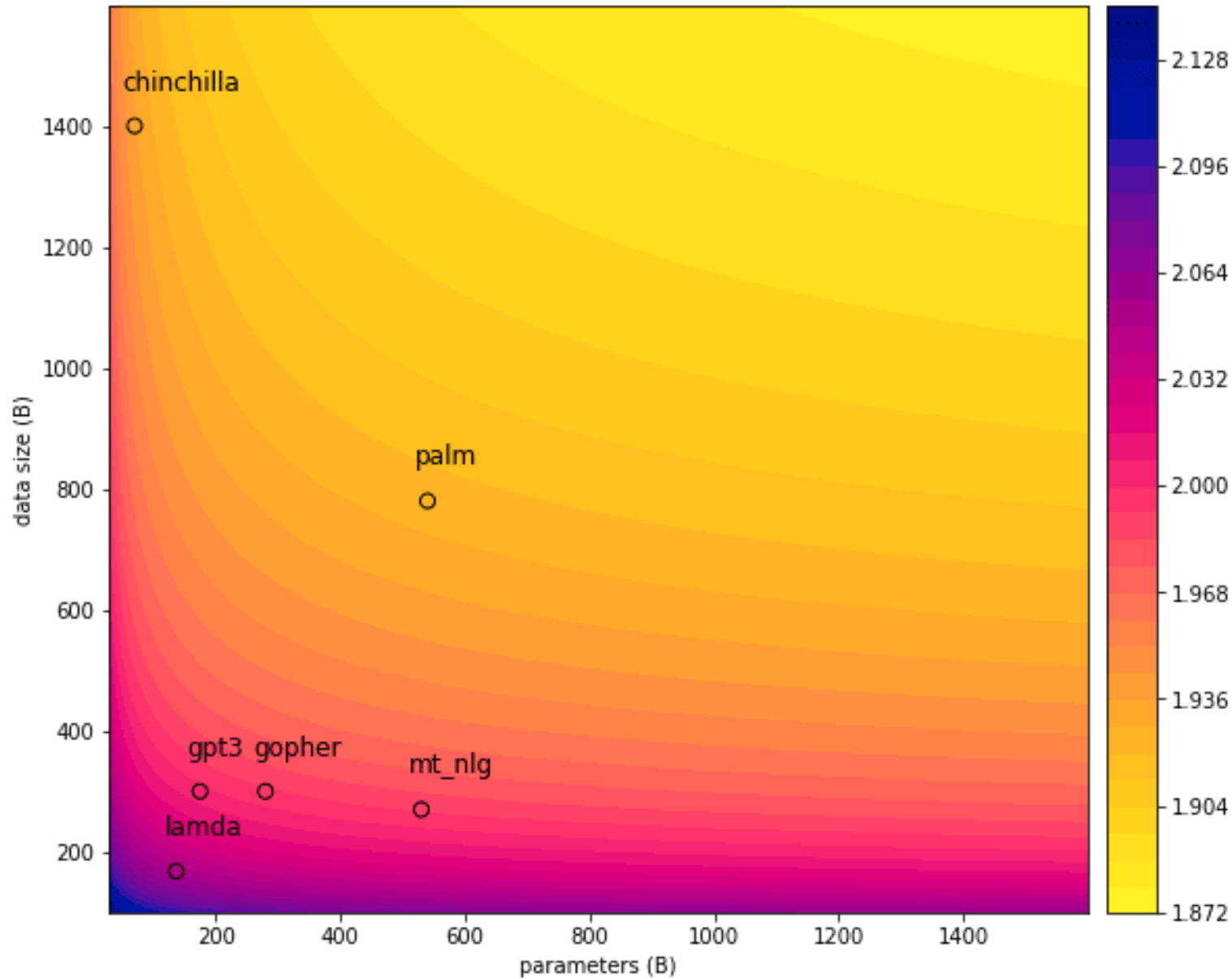
Training Compute-Optimal Large Language Models

For optimal scaling,
we must increase
all:

- Model size
- Compute budget
- DATA



Andrew Ng summary: “Data is the new oil”

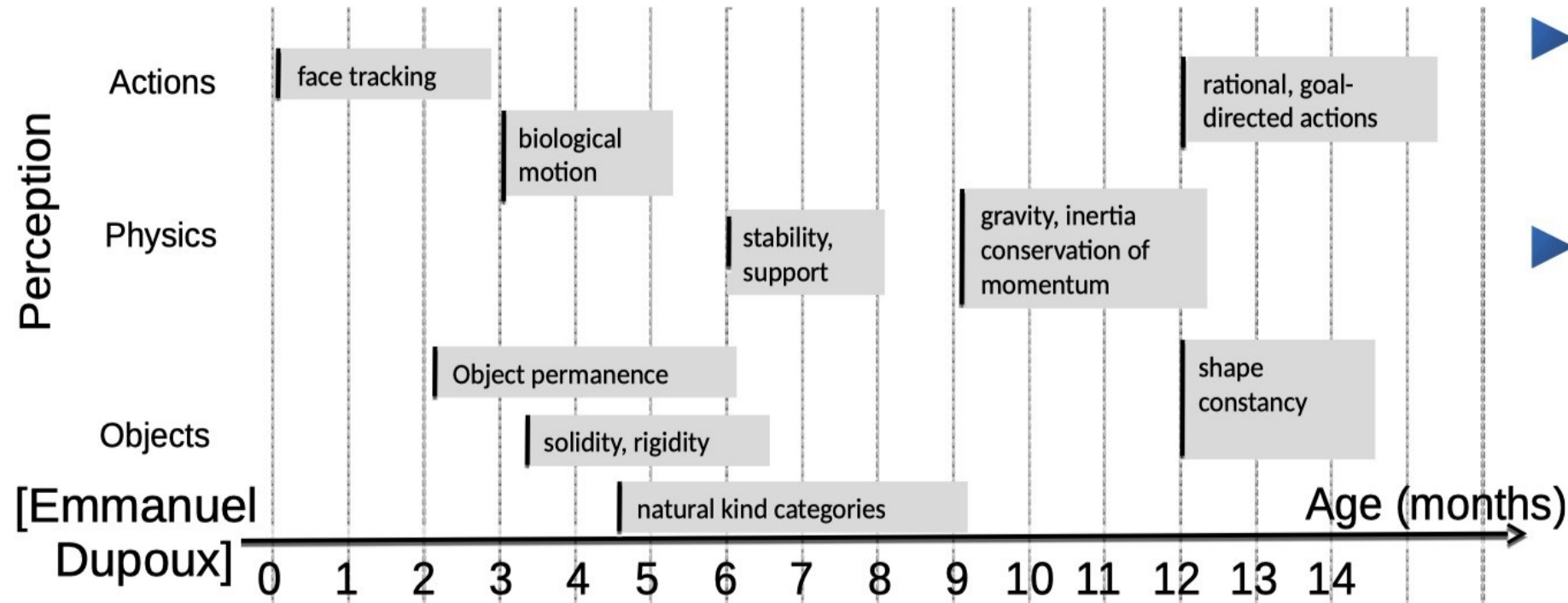


Contour colors represent loss

“Chinchilla” gets better loss than gpt3 et al with far fewer parameters, by using more data

(Phi-2 model goes this route)

How could machines learn like animals and humans?



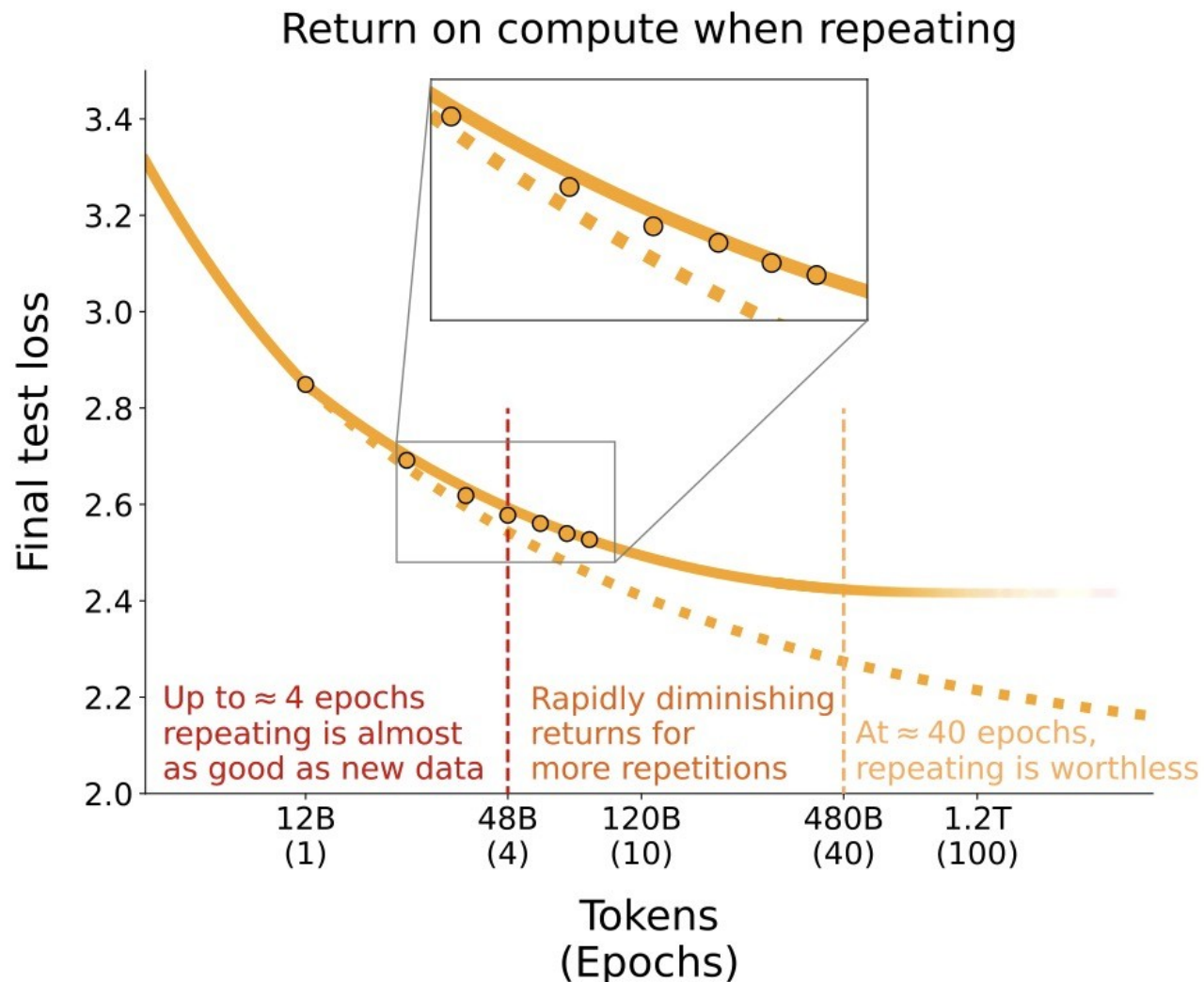
- ▶ How can babies learn how the world works?
- ▶ How can teenagers learn to drive with 20h of practice?

- Aren't humans much more efficient, at least in data?
- Or evolution is a way to "scale" data (and compute)?

Scaling Data-Constrained Language Models (NeurIPS 2023 Outstanding paper runner-up)

- Will we run out of data?
<https://arxiv.org/pdf/2211.04325.pdf>
Yes, especially for low resource languages
- How much can we wring out of existing data?

Reasoning, RL,
multimodal are the



Scaling LLM Test-Time Compute Optimally can be More Effective than Scaling Model Parameters

Charlie Snell^{◆, 1}, Jaehoon Lee², Kelvin Xu^{♣, 2} and Aviral Kumar^{♣, 2}

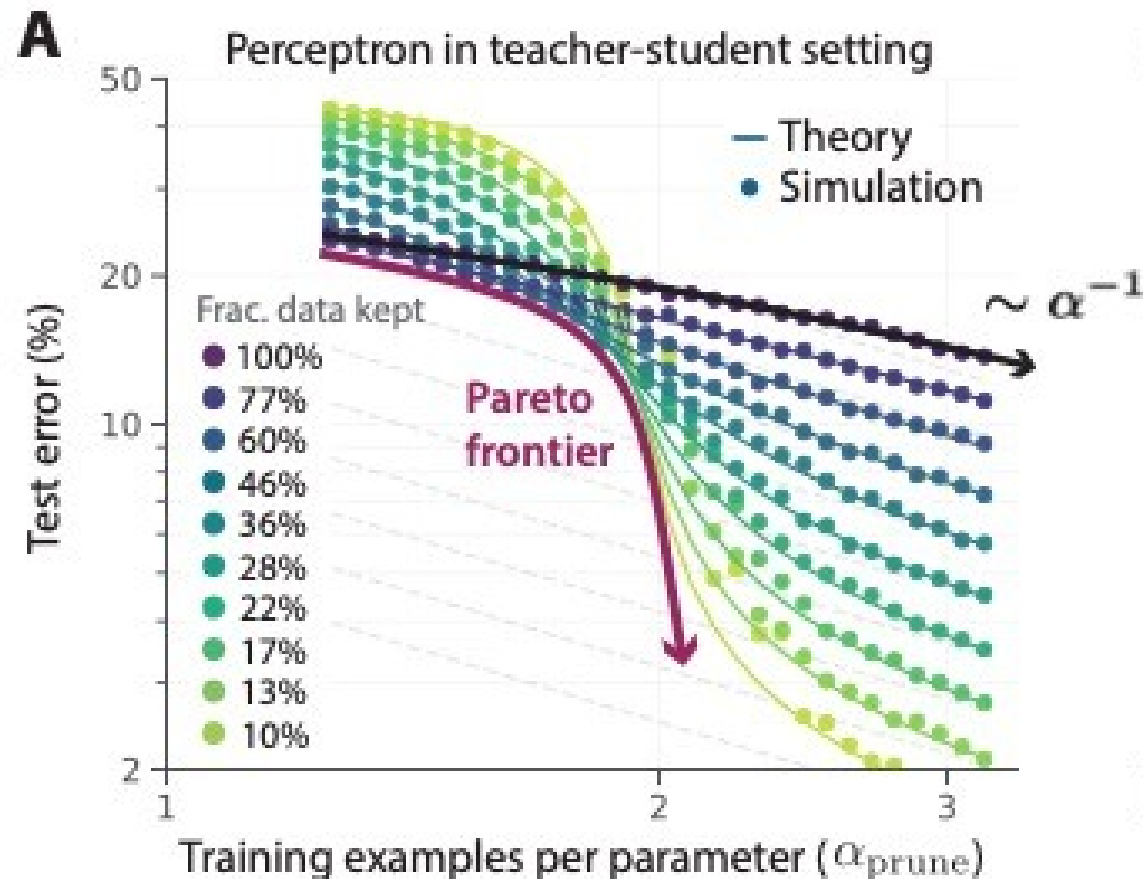
♣Equal advising, ¹UC Berkeley, ²Google DeepMind, ◆Work done during an internship at Google DeepMind

Do more pretraining, or more “test time compute?”

Test time compute =

Chain of thought, or
Generate multiple answers
and choose best

Beyond neural scaling (NeurIPS 2022 Outstanding paper award)



- Prune easy examples as data size grows
- Improvements for LLMs, but not as big as for small models in paper (see [arXiv:2404.07177](https://arxiv.org/abs/2404.07177))
- Data filtering is probably one of the greatest “secret sauces” of the Frontier

Other directions

- Improve attention
- Diffusion language models (will discuss later in class)
- Reasoning + RL, alignment, tuning
- AI for science
- Agents
- Mechanistic interpretability (small)

TurboQuant – Compress the KV cache

Save space by quantizing (regular quant)

+0.738925792	->	+0.73
-0.832182459	->	-0.83
+0.132467954	->	+0.13

TurboQuant

Problem... nearly sparse vectors lose most info

+0.999654324	->	+0.99
-0.000455344	->	-0.00
+0.000042434	->	+0.00

TurboQuant

Solution: randomly rotate vectors (random rotation matrix)

Then quantize

+0.999654324 -> +0.738925792 -> +0.73

-0.000455344 -> -0.832182459 -> -0.83

+0.000042434 -> +0.132467954 -> +0.13

To recover, apply inverse rotation – usually preserves more bits in original basis

+0.73 -> +0.999652

-0.83 -> -0.000457

+0.13 -> +0.000043

Next time – start modern
paradigms in computer
vision